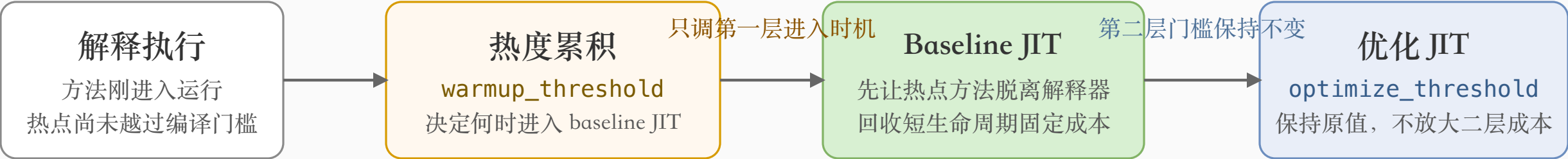


方案一：JIT 分层时机与阈值调整

调参规则
LoongArch64 + debug + !JavaDebuggable + !explicitWarmup => warmup = 8192



为何命中 `pmd`
生命周期短，热度窗口有限
若 warmup 门槛过高，热点方法来不及进入 baseline JIT
收益首先体现为解释器驻留时间下降
正文最具代表性的结果是 `DaCapo/pmd`

为何不直接动第二层
优化重编译阈值只影响更晚的阶段
短生命周期 workload 的首要问题仍是“第一层来不及发生”
过早放大第二层活动，反而可能增加固定编译成本
因此正文正式方案限定为“只调 warmup_threshold”

结论定位
方案一回答的是“LoongArch64 debug ART 的第一层 JIT 进入时机是否偏晚”，
因此它更适合解释短生命周期 workload 的受益方向，而不是作为所有 JIT 层级参数的统一默认改写。